

Welcome to Week 1 of CS 5200

Database Management Systems · Summer 2026

Hi, I'm Dan

ALIGN graduate (MS in CS - 2024). I sat where you are sitting two years ago.

Before the Roux: 10+ years in the technology industry as a journalist and then working in public relations.

Current: Software developer at IDEXX, previously worked as a developer at Tyler Technologies

Teaching: TA for multiple courses while a Roux student. I also teach an Introduction to Programming in C course at the University of Southern Maine.

On the side: building a software startup that is going to begin beta testing with a town in Maine in the coming weeks

Teaching Style

I love learning new things but I don't know everything.

There will be times when I can't answer a question, and I will admit it.

I may talk too fast, mispronounce a word, or mumble. I try my best to communicate in a clear way for all to understand, but if I ever need to repeat something or provide more examples, please tell me.

My greatest asset is my ability to merge my experience as a student and an industry professional.

Today

Course logistics and the syllabus

What is a database?

Why bother learning databases?

The relational model and PostgreSQL

Live demo: DBeaver and Pagila

Start work on Lab 1

Homework 1 Introduction

Tell Me About Your Classmate

Quick round of introductions. When it's your turn, tell me six things about your classmate:

- Name
- Hometown
- Program they are enrolled in (MSCS, MSAI, MSDS, ALIGN other) and whether they are working full-time alongside it.
- What brought them to the Roux / What did they do before? (Undergrad, working, something else...?)
- One thing they hope to learn this semester. Could be technical, could be career-oriented, could be "just survive the term." Honest answers welcome.
- Something fun about them or something fun they do.

Course Logistics

Class norms

Ask a question anytime.

Raise your hand if I am speaking, otherwise feel free to simply ask when there is a pause.

Classes are live streamed but not recorded.

Make sure to always bring your laptop.

How to Reach Me

Canvas announcements are the official channel for schedule changes and assignment clarifications. Check before class each week.

Email (d.graziano@northeastern.edu) is best for one-off questions and any kind of accommodation request. 24-hour response on weekdays. Response times on weekends are inconsistent. If something's urgent, get it to me by Friday afternoon.

Office hours (Tuesdays 5:00–6:00, this room): drop in, no appointment, no agenda required. Use them. Even just to say hi.

Zoom by appointment: I have two or three evenings open most weeks outside of class. Email to set one up.

Rule of thumb: if you've been stuck on something for an hour, the right next step is reaching out through any of these channels. Don't sit on it.

Expectations

You have had at least one programming course (probably Python).

You can write a program on your own and can also program as part of a team (at least with one other programmer).

You can read/skim technical material that is beyond your current expertise - reading in detail until you get stuck, then skimming over the detailed stuff.

You have little to no experience with databases.

Required Software

Everything we use this semester is free. A detailed install guide is available on Canvas.

PostgreSQL 18 - installed locally. This is our database engine for the entire course.

DBeaver Community Edition - our recommended GUI client for browsing schemas and running queries.

psql - the command-line client bundled with Postgres. You will use both.

Python 3.11+ with psycopg - needed for the Week 11 application unit. Plenty of warning before then.

A diagramming tool of your choice for ER diagrams - dbdiagram.io, draw.io, or pen-and-paper all work.

Before You Leave Tonight

- ☐ PostgreSQL 18 installed and running on your laptop
- ☐ DBeaver Community Edition installed and connected to your local Postgres
- ☐ Pagila loaded as a database, restorable from the dump file on Canvas
- ☐ You have run a real SELECT against it (we'll do this in Lab 1)

*If any of these are still not working by next Tuesday, email me before class. We'll fix it together.
Nobody should start Week 2 without a working setup.*

Books & Reading

Required: the official PostgreSQL 18 documentation at <https://www.postgresql.org/docs/18/>. It is genuinely one of the best technical manuals in software. Free, online, exhaustive.

Recommended: Learning SQL, 3rd ed. by Alan Beaulieu. A gentle, exercise-backed primer that uses a sample database close to ours. Free with your Northeastern email through O'Reilly.

Recommended: SQL Performance Explained by Markus Winand. Short, sharp, and free at use-the-index-luke.com. Assigned reading for Week 10.

No textbook purchase is required. I will hand you readings and links throughout the term.

YouTube is also a great resource - Andy Pavlo's CMU lectures and Hussein Nasser's channel are good free supplements.

Policies

Attendance isn't taken but you are expected to be here. There is a participation grade.

You may not copy anyone else's homework or code under any circumstances. This includes online sources and LLMs (more on that later)

You may not let any other student see any part of your program.

You may not view any part of another student's work.

Anything submitted must be in the format of CS5200_Assignment_Lastname. For example CS5200_HW1_Graziano.pdf.

All submission files must include a top header with your name, date, and assignment. Points will be deducted if missing.



Grading

Component	Weight	Notes
Homework (~10 assignments)	35%	Roughly weekly. Each assignment builds on the previous one.
In-class labs / participation	10%	Graded on attendance and good-faith effort
Take-home midterm (Week 7)	20%	Distributed end of Week 7, due one week later. No in-class exam.
Final project (Week 13)	25%	Database-centered. Schema freeze checkpoint in Week 9.
Quizzes	10%	5–6 short take-home quizzes. Open after class, due before the next.

An 'A' on a homework requires depth of understanding. A 'C' is technically correct but no more. An 'F' has too many flaws to tell.

How I Grade Homework

Homework is graded out of 100. The letter equivalents are as follows:

A - answers that show depth of understanding. You explained what you did, why, and what the tradeoffs were. The kind of submission I'd happily forward to a teammate.

B - solid, technically correct, well-explained. The default for good work. Many submissions land here.

C - technically correct and nothing more. The query runs but no judgement is offered.

F - too many flaws or blank responses for me to tell whether the learning landed.

The key thing I am looking for is evidence of thought. You should be putting actual time into these assignments.

Quizzes

5 to 6 short take-home quizzes spread across the semester. Each opens at the end of a class meeting and closes before the next one.

Each quiz is 5–10 questions, mostly conceptual, some short SQL. Designed to take ~30 minutes if you've been keeping up.

Closed-book with respect to AI. No LLMs like ChatGPT or Claude. The point is to check what's actually in your head.

Hard deadlines. No grace window, no Time Travel Days. If life happens, email me before the deadline.

Together they're 10% of your grade - small individually, but worth enough that you shouldn't skip them.

Time Travel Days

You get six (6) no-fault Time Travel Days for the semester. Use them to extend any homework deadline by up to three days, no questions asked.

How to use one: email me before the deadline with at least one substantive question about the assignment. That tells me you've actually started it.

If you don't use them: late submissions lose 10% per day, up to 3 days. After that, no credit without prior arrangement.

No grace window on quizzes, the midterm, or the final project. Plan accordingly.

Real engineering teams accommodate the occasional missed deadline; they don't accommodate chronic ones. This course tries to do the same.

Note that time travel days and the late penalty cannot be combined on the same assignment.

Stuck for an Hour? Stop.

This is the single most useful habit you can develop in this course.

If you've been on the same problem for an hour with no progress: stop. The next thing you should do is reach out - not push through.

Email me. Ask a classmate. Go for a walk. Come to office hours.

Spinning your wheels at 11 PM the night before a deadline is the worst possible use of your time. You're tired, you're frustrated, and you're not learning anything.

Real engineers do not solve every problem alone. Neither should you.

Ask Questions!

It makes class more interesting.

It helps you and others learn.

You will need to ask questions (A LOT) in your career. Why not start now?

There's no such thing as a stupid question....

But if you feel intimidated, use the anonymous Google Sheet

https://docs.google.com/spreadsheets/d/1d8P4_1X2HK81diynlCluyDY_dZuu_SSLV_M_me7vfvQE/edit?usp=sharing

AI Tools & Code Generators

Large language models - ChatGPT, Claude, Copilot, Cursor - are real software-engineering tools. You will use them in your career. This course does not pretend otherwise.

And yet: the work that makes you a better engineer is writing your own SQL, debugging your own errors, and justifying your own design choices. Not what the LLM produces while you watch.

Mistakes are the best teacher I know. Your imperfect, in-your-own-voice answer beats a polished generic one every time.

The policy on the next slide tries to be honest about what helps you learn - and what doesn't.

AI Tools - The Rules

Allowed

Asking an LLM to explain a concept, an error, or unfamiliar SQL.

Autocomplete tools (Copilot or Cursor) for routine syntax while you write your own logic.

Asking for a review after you have a working solution.

Not Allowed

Submitting LLM-generated work as your own without substantive engagement.

Outsourcing your project schema, your normalization decompositions, or the bulk of your homework solutions.

Using an LLM during quizzes or the take-home midterm.

If you can't explain every line of what you submitted, it isn't your work.

No one cares about the piece of paper you will get from this program. They care about the knowledge you gain from it.

You won't gain that knowledge without the struggle.

LLMs have changes education. I saw this as a student, then as a TA, and now as a teacher. They are incredible tools but don't allow you to retain information like doing the work yourself does.

AI Tools - Disclosure

Every homework submission that uses an LLM gets a one- or two-sentence note at the top of the file. That's a requirement.

```
-- I used Claude to debug a syntax error in problem 4.  
-- I used ChatGPT to explain how window functions work  
-- before attempting problem 7.  
  
-- Otherwise, all queries below are my own.
```

Honest disclosures cost you nothing. Submissions without disclosure that look LLM-generated are an academic-integrity issue. Also, AI submission look strikingly similar to other students' submissions, which makes the lack of original work obvious.

Two More Ground Rules

Stuff Happens Clause

Illness, family emergency, work crisis. Email me as soon as you reasonably can. I do not need details - but I do need to hear from you, ideally before the deadline. We'll work out a plan.

Collaborate, don't copy

Discussing problems with classmates is encouraged. Copying their code or answers is not.

A good test: you should be talking *English with one another, not SQL*.

Our Weekly Rhythm

6:00–6:12	Opening recap - quick retrieval of last week's material
6:12–6:50	Lecture block 1 - the core idea of the week
6:50–7:00	Break
7:00–8:00	Lecture block 2 - live demo or guided example
8:00–8:10	Break
8:10–9:10	Lab - hands-on keyboard time
9:10–9:20	Wrap-up, homework framing, preview of next week

This is the shape of every class from Week 2 onward. Today (Week 1) we deviate slightly because of first-day logistics.

Where We're Going

Wk 1	Welcome, Postgres, Pagila tour	today
Wk 2	ER modeling + soccer league	
Wk 3	Keys, constraints, DDL	
Wk 4	SQL fundamentals - filtering, sorting, aggregation	
Wk 5	Joins, CTEs, window functions	
Wk 6	Normalization + functional dependencies	
Wk 7	Relational algebra + project pitches	midterm
Wk 8	Functions, triggers, server-side logic	
Wk 9	Transactions, ACID, concurrency	schema freeze
Wk 10	Indexes, B-trees, query plans	
Wk 11	Security + Python integration	
Wk 12	Specialized engines	
Wk 13	Migrations + the modern data ecosystem	project due
Wk 14	Finals week – Presentation	

Your Homework Domain

Starting next week, you'll design and build a database for a soccer league. Teams, players, coaches, matches, goals, cards, venues, seasons. The whole thing.

Same domain for the rest of the semester. Each week's homework adds another piece to it.

HW2: the design. HW3: the schema. HW8: functions and triggers.

You learn the techniques without also learning a new business context every week. By Week 11, you'll know this schema better than your own apartment.

Your Final Project

Database-centered. You pick the domain (not the soccer league - something you actually care about) and deliver:

An original domain, an ERD, a normalized schema, sample data, 10+ meaningful queries, at least one function or trigger, at least one index with written justification, and a 2–3 page write-up defending your design choices.

Application integration is optional. If you want to wrap your Week 11 Python CLI into something richer, go for it. Not required for full credit. The database is the center of gravity.

Schema freeze in Week 9. After that, design changes need my approval. This is to save you from yourself - students always want to redesign the schema two days before submission.

Due Week 13. Presentations Week 14.

Some Helpful Advice



It may feel like you are drinking from a firehose

Absorb some of what I say

Get the rest through practice labs and homework assignments.

What is CS 5200?

CS 5200 introduces relational database management systems as a class of software systems and prepares students to be sophisticated users and basic administrators of a modern RDBMS. The course covers design theory, query language, performance tuning, and the architecture and ACID properties of relational systems. Topics include entity-relationship modeling, normalization, SQL, relational algebra, server-side programming with functions and triggers, transactions and concurrency, indexing and query plans, security, and client-server integration. The course closes with a survey of specialized engines (document, spatial, full-text, time-series) so students leave with a map of where the relational core fits in the broader data landscape.

This is the course description from Northeastern and the one included on the syllabus.

It translates to an introduction to databases... and more specifically an introduction to relational databases, which use SQL as the language to interact with them.

What is a database?

A database is...

Something that stores data....?

But how is that different from a text file or spreadsheet?

When the Spreadsheet Breaks Down

Imagine you're tracking your soccer league in a spreadsheet. One row per game. It works fine for a Saturday.

Now: thirty teams. Three seasons of history. Two people editing at once. A coach moves between teams. A player's name was misspelled - in 47 places.

The spreadsheet starts to lie to you. Different versions disagree. Edits overwrite each other. "The data" no longer has a single answer.

A database is what you reach for when the spreadsheet breaks down - and that point comes earlier than you'd think.

Here's What That Looks Like

Player	Team	Goals	Date Joined
Smith, Joe	Riverside FC	5	4/5/22
Joe Smith	Riverside	5	April 5, 2022
smith, joe	Riverside F.C.		2022-04-05
Anne Jenkins	Pinewood United	12	10/8/2023
Anne Jenkins	Pinewood Utd	12	Oct 2023
JENKINS, ANNE	pinewood united	12	

Same player, same team - three different spellings of each, two date formats, and a missing value. Now imagine 30,000 rows. The spreadsheet stops being a record of truth and starts being a record of disagreements.

A database is....

An organized collection of related data, stored so that it can be efficiently retrieved, updated, and shared.

They are usually in some predictable structure (rows and columns, documents, key-value pairs).

This let's you find things efficiently rather than scanning everything.

A more official definition of a database is an organized, electronically stored collection of structured information or data, typically managed by a Database Management System (or a DBMS)

Three key words

Organized - there is structure. Not a pile of files.

Related - a database covers a connected slice of the world (one league, one store, one company).

Shared - multiple users and applications can access it at once without stepping on each other.

Some Vocabulary

Schema - the structure: which tables exist, what columns they have, what types those columns hold, what constraints apply.

Table - a named collection of rows with a fixed set of columns. (Think: one sheet in a spreadsheet, but with rules.)

Row - a single record. One player. One game. One rental.

Column - a named, typed field that every row in a table has.

And One Last Word

Query - a question you ask the database, written in SQL.

It's a string of text. You hand it to the database software and you get back a result.

```
SELECT name, goals
FROM   players
WHERE  goals > 5
ORDER BY goals DESC;
```

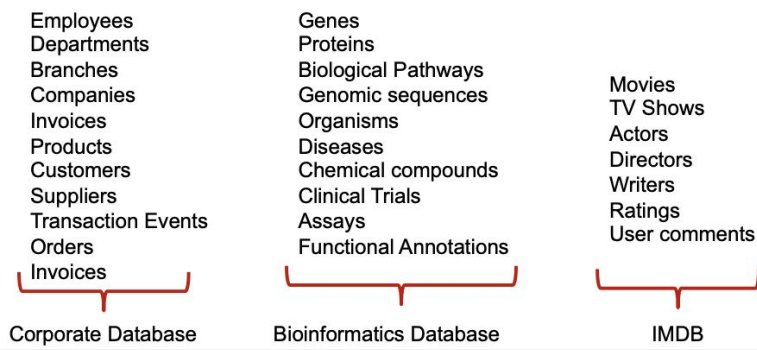
We will write hundreds of these. By Week 4, writing one will feel like writing a sentence.

Modeling the real world

A database is a model of real world entities, attributes, and relationships.

Each particular database carves out a domain of connected or related entities.

Entities are the things we want to model and can include things like:



A Tiny Database

Here's a small players table. 5 columns × 4 rows.

id	name	team_id	position	goals
<i>INT PK</i>	<i>TEXT</i>	<i>INT FK</i>	<i>TEXT</i>	<i>INT</i>
1	Maya Hernandez	12	FW	8
2	Tariq Williams	12	MF	3
3	Sam O'Brien	07	GK	0
4	Aiko Tanaka	07	DF	1

The header row plus the type annotations is the schema. Each data row is a record. The whole thing is a table - and a real database is many tables, with rules connecting them.

Database Management System

A database management system (DBMS) is the software that sits between you and your data.

It does the hard work that you don't want to do yourself:

- Stores data on disk in a way that's fast to read and write.
- Enforces the rules of your schema - types, keys, constraints.
- Coordinates concurrent access from many users without corruption.
- Recovers from crashes without losing your data.

In short, it's a software system that enables users to define, create, maintain, and control access to the database.

PostgreSQL, MySQL, Oracle, SQL Server, SQLite - all DBMSs.

Where the DBMS Sits

*Python, Go, JS, Java...
whatever talks to the database*

Your application code

↓ *SQL queries* ↓

DBMS (PostgreSQL)

query planner · transaction manager · cache · locks

*Validates types, enforces constraints,
coordinates concurrent users,
recovers from crashes.*

↓ *reads & writes* ↓

Disk (your data, on storage)

Your code never touches the disk directly. Every read and every write goes through the DBMS - which is what lets ten people connect at once without corrupting each other's work.

Database vs. DBMS

Database

the data itself

Tables, rows, indexes, constraints. The rentals, the films, the customers.

Sits on disk. Doesn't do anything by itself.

DBMS

the software that runs it

Stores, retrieves, updates, validates, locks, backs up, recovers.

Postgres, MySQL, SQL Server, SQLite are all DBMSs.

Casual usage blurs them - "my Postgres database." In this course we'll often be precise.

Advantages of DBMSs

Control of data redundancy

Sharing of data

Improved data integrity

Improved security

Enforcement of standards

Handles concurrent access safely

Offers durability and consistency guarantees

Queryable - There's a language or API for asking questions like "give me all customers in Maine who ordered last month"

Disadvantages of DBMSs

Complexity

Size

Cost of DBMS

Additional hardware costs

Cost of conversion / vendor lock-in

Performance bottlenecks

Many of these problems become magnified at “big data” scales

Higher impact of a failure

Why learn databases?

They Are Everywhere

Every meaningful application you've used today is sitting on top of one - usually several.

Banking

every transfer, every balance

Healthcare

EMRs, claims, lab results

E-commerce

products, carts, orders, returns

Social

your feed, follows, messages

Logistics

package tracking, fleet, routing

Streaming

catalogs, watch history, recs

Government

tax records, voter rolls, permits

Science

genomes, telescopes, climate

Your phone

iMessage and Photos run on SQLite

A Day in My Work

Things I did at Tyler Tech / IDEXX in a single sprint, all of which are database work:

- Restored a customer's database and wrote multiple queries to explore for potential problems
- Optimized a slow query that was timing out the dashboard. Added one composite index to reduce the time from 12s to 80ms.
- Wrote a migration to add a column with backfilled values for ~1 million existing rows.
- Discussed with colleagues about whether a new feature should be stored as a JSON blob or a real table. Real table won.

None of those is a "database administrator" job. They're software engineering. Every backend engineer ends up doing this kind of work.

Why You Should Care

It's the most reused thing you'll learn this year. JavaScript frameworks come and go. SQL has been around since 1974, and it isn't going anywhere.

Every backend engineer is a part-time database engineer. You don't get to opt out - the question is whether you do it well.

Bad schemas cost real money. Many production performance problems aren't the language or the framework. They're the database - a missing index, a wrong join, an inefficient query.

Interviewers ask. SQL is on almost every backend, data, and full-stack interview I've seen, mine included.

By End of This Course

In 14 weeks, you'll be able to:

- Read a real-world domain and produce a normalized relational schema.
- Write SQL ranging from simple lookups to multi-table aggregations using joins, subqueries, CTEs, and window functions.
- Apply normalization and recognize when denormalization is the right call.
- Implement server-side logic with functions and triggers in PL/pgSQL.
- Reason about ACID, transaction isolation levels, and concurrency control.
- Read a query execution plan and choose indexes that actually help.
- Wire a relational database into a Python application via psycopg, safely (parameterized queries, transactions).
- Defend your design choices in writing, with the tradeoffs articulated.

And in Interviews

These are real questions I and my colleagues have asked candidates over the past few years:

"Walk me through a JOIN you wrote recently. Why that join type?"

"How would you index this query? What's the tradeoff?"

"What happens if two users update the same row at the same time?"

"How would you design a schema for our app"

"What's the difference between a foreign key and a primary key?"

After this course you will be able to answer all of them.

Some roles that use databases

Database administrator (DBA)

- Owns the operational health of the database. Deals with backups and restores, permissions/security, performance tuning, uptime, etc...

Software developer / application developer

- Uses databases to store and retrieve application data, which involves writing queries, handling migrations, and API integrations.

Database developers

- Write complex queries and functions for databases,

Data analyst

- Uses databases to answer business questions with queries and dashboards

Data engineer

- Build the pipelines that move, clean, transform, and organize data.

What is a RDBMS?

This stands for Relational Database Management Software.

It's the software used for relational databases, which are the types of databases we will focus on in this course.

Before Relational

Three flavors of "database" existed before 1970, and each had real problems.

Flat files. Data scattered across application-owned files. No relationships. No validation. No concurrency story. Two programs editing the same file = corruption.

Hierarchical (1960s). IBM IMS. Data is organized like a tree. - a parent record with child records. Great if your domain is actually tree-shaped (rare). Painful otherwise.

Network (1969, CODASYL). Data as a graph. To find a record you traversed pointers from one record to the next. Programs were tightly coupled to the layout - change anything and the code breaks.

All three forced your code to know how data was stored on disk. That coupling was the real problem.

Example of the File-Based Approach

```
numbers = {}

in_file = open("user_values.txt", "rt")

while True:
    in_line = in_file.readline()

    if not in_line:
        break

    in_line = in_line.strip()

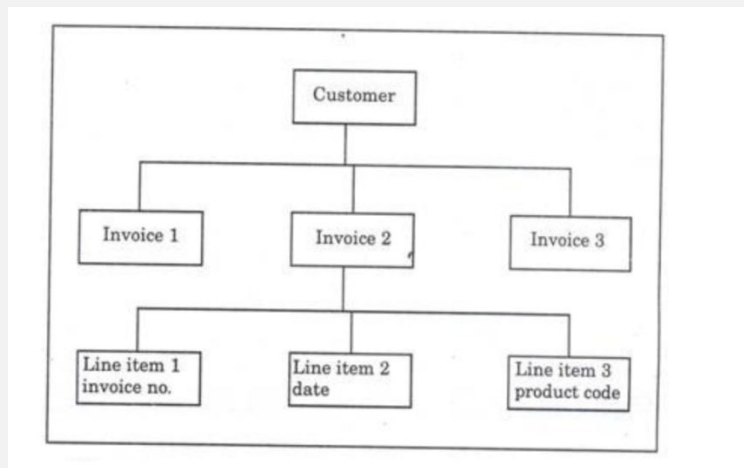
    name, number = in_line.split(",")

    numbers[name] = number

in_file.close()
```

This code is reading a text file line by line, splitting each line into a name and a number, and storing them in a dictionary called numbers

Hierarchical Data Model



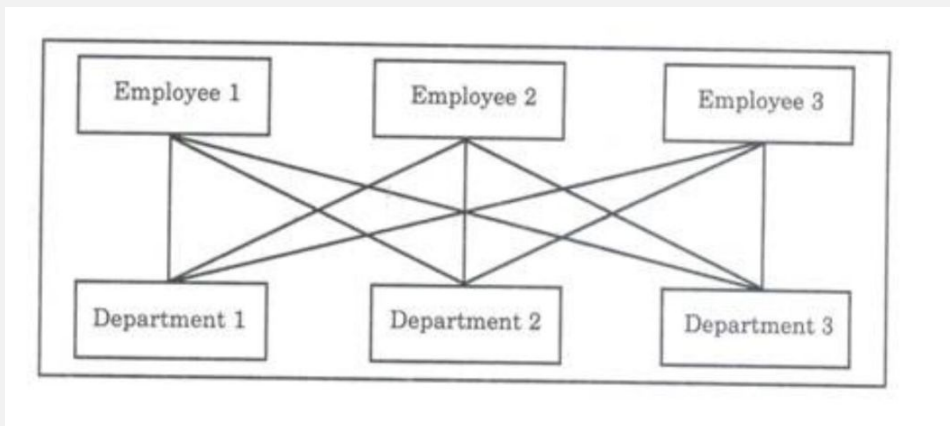
Each record has a parent, and each parent can have many children. But each child usually has only one parent

They were not very flexible. The programmer needed to know the database's structure very well. Otherwise, they had to get the parent, then look at each child until they found what they were looking for.

Good for when the data naturally forms a tree and the main access pattern follows that tree. For example: a file system or an org chart.

These were bad for real-world data. For example, one student can take many courses and one course can have many students. That is a many-to-many relationship that would require duplicate data (a tree for the student that contains courses they are taking and a tree for the courses with the student in them).

Network Data Model



Network databases were more flexible than hierarchical databases because a record could have multiple parent-like connections, rather than being limited to one parent.

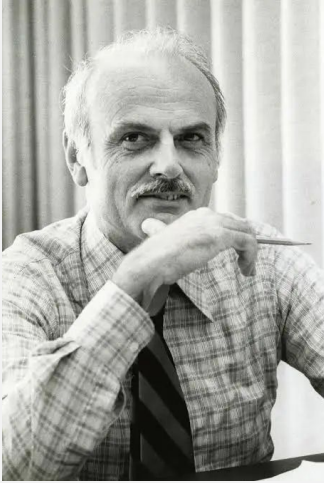
This allowed them to represent many-to-many relationships, such as students and courses, more naturally than a strict hierarchy. However, they could still be difficult to navigate because programmers needed a strong understanding of the database model and its predefined access paths.

For example, a program might start at a known entry point, such as a specific Student record, then follow that student's enrollment links to the related Course records one at a time.

History of Databases

1970	E.F. Codd (IBM) and the relational model
1970's	Early relational database implementations
1976	The Entity-Relationship Model: A tool for designing databases
1979	Oracle database released, followed by DB2 by IBM in 1983
1986	SQL Standards emerge (there are still many dialects)
1990's	The growing importance of data-mining, machine learning, and the Internet
1995	MySQL
2000's	Beyond the relational database, Big Data, NoSQL
2008	Sun Microsystems acquires MySQL
2010	Oracle acquires Sun Microsystems
2010's	Non-Relational (NoSQL) databases proliferate, but relational DB's continue to dominate.

Codd's Big Idea (1970)



Edgar F. "Ted" Codd worked at IBM in the late 1960s.

Databases were navigational - to find your data, you had to know the path. Changing the storage format meant rewriting applications.

He proposed to model your data as relations (tables of rows and columns), and let users ask questions declaratively - say what you want, not how to get it.

Two ideas, one paper. They turned out to be the foundation of every relational database in use today.

The Big Win: Data Independence

Before relational: your code had to know exactly how the data was stored. The DBMS reorganized? Your application broke.

After relational: you declare what you want. *"Give me the names of customers in Portland."* The DBMS figures out how to fetch them - sequential scan, index lookup, hash join, doesn't matter to you. The DBMS is free to change its internal layout, add indexes, switch storage formats. Your code keeps working.

This separation between *what you ask* and *how it's executed* is called data independence. It is the single most important thing the relational model gave us.

It's also the reason you've used databases your whole life without ever reading a chapter on B-trees (self-balancing tree)

The Relational Model

Data lives in tables. Each table has a fixed set of columns and any number of rows.

Relationships are values, not pointers. If a player belongs to a team, the player row carries the team's ID - not a pointer to its memory address.

Order doesn't matter. Rows and columns have no inherent order. The DBMS is free to store and retrieve them however it wants.

You ask declaratively. "Give me players older than 30, sorted by goals scored." The DBMS figures out how.

That's it.

A Simple Relational Data Model

Name	Salary	Branch
John White	30000	22 Deer Rd, London
Ann Beech	12000	163 Main St, Glasgow
David Ford	18000	163 Main St, Glasgow
Mary Howe	9000	16 Argyll St, Aberdeen
Susan Brand	24000	163 Main St, Glasgow
Julie Lee	9000	22 Deer Rd, London

What are the disadvantages of this design?

Data redundancy... we have multiple repeated addresses. This is wasted storage space.

A Better Relational Data Model

Branch

branchNo	street	city	postCode
B005	22 Deer Rd	London	SW1 4EH
B007	16 Argyll St	Aberdeen	AB2 3SU
B003	163 Main St	Glasgow	G11 9QX
B004	32 Manse Rd	Bristol	BS99 1NZ
B002	56 Clover Dr	London	NW10 6EU

Staff

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

Why is this better?

It separates the data into two related tables. Notice how we use a key to reference the data from one table to the other.

Another example from earlier

Here's a small players table. 5 columns × 4 rows.

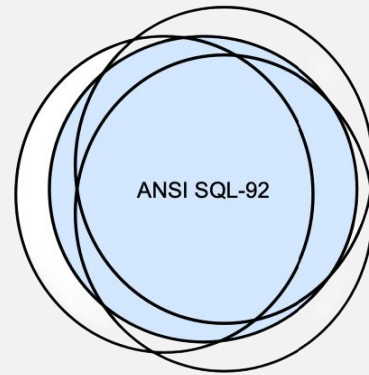
id	name	team_id	position	goals
<i>INT PK</i>	<i>TEXT</i>	<i>INT FK</i>	<i>TEXT</i>	<i>INT</i>
1	Maya Hernandez	12	FW	8
2	Tariq Williams	12	MF	3
3	Sam O'Brien	07	GK	0
4	Aiko Tanaka	07	DF	1

SQL: The language of databases

SQL = Structured Query Language

Pronounced either S.Q.L. or See-quel (both are perfectly acceptable).

The language was standardized over time (SQL-86, SQL-89, SQL-92) but most vendors do not implement the standard completely or they introduce vendor-specific features / dialects



One standard, many dialects

Why SQL Won

SQL is the language we use to talk to relational databases. IBM built it in the 1970s. Oracle commercialized it in 1979.

It's relatively simple to learn, although difficult to master.

Half a century later, the same five core verbs run the world's data:

SELECT

INSERT

UPDATE

DELETE

CREATE

That's the entire SQL skeleton. Everything else is options on these verbs.

ANSI stands for the American National Standards Institute. It's a non-profit organization that oversees the development of voluntary consensus standards for products, services, processes, and systems in the United States.

A Taste of SQL

Read this query out loud in plain English. You'll find you almost can already.

```
SELECT    customer_id, SUM(amount) AS total_spent
FROM      payment
WHERE     payment_date >= '2007-01-01'
GROUP BY  customer_id
ORDER BY  total_spent DESC
LIMIT     10;
```

"Top 10 customers by total spend, since the start of 2007." By Week 4 you'll write this in your sleep.

SQL Basics

SELECT	<Columns to retrieve>
FROM	<Source tables>
WHERE	<Conditions on individual rows>
GROUP BY	<Collapses multiple rows with a shared value>
ORDER BY	<Sorting criteria>
LIMIT	<Restricts the number of records returned>

```
SELECT    customer_id, SUM(amount) AS total_spent
FROM      payment
WHERE     payment_date >= '2007-01-01'
GROUP BY  customer_id
ORDER BY  total_spent DESC
LIMIT     10;
```

There are a lot of databases to choose from

Rank			DBMS	Database Model	Score		
May 2026	Apr 2026	May 2025			May 2026	Apr 2026	May 2025
1.	1.	1.	Oracle	Relational, Multi-model ⓘ	1143.28	-14.66	-83.29
2.	2.	2.	MySQL	Relational, Multi-model ⓘ	856.49	-1.20	-108.49
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model ⓘ	700.99	-1.08	-73.90
4.	4.	4.	PostgreSQL	Relational, Multi-model ⓘ	682.68	+1.33	+8.37
5.	5.	5.	MongoDB	Document, Multi-model ⓘ	384.65	-0.37	-17.87
6.	6.	6.	Snowflake	Relational	215.64	-1.73	+43.63
7.	7.	↑ 12.	Databricks	Multi-model ⓘ	151.79	+1.31	+49.13
8.	8.	↓ 7.	Redis	Key-value, Multi-model ⓘ	147.63	+1.09	-4.56
9.	9.	↓ 8.	IBM Db2	Relational, Multi-model ⓘ	113.04	-0.21	-13.36
10.	10.	↑ 11.	Apache Cassandra	Wide column, Multi-model ⓘ	101.67	-0.95	-6.39

via DB-Engines Ranking

Why PostgreSQL?

Free and open-source. You can run it on your laptop, on AWS, on a Raspberry Pi. No license, no vendor lock-in.

Standards-compliant. More compliant with ANSI-SQL standards than major competitors like MySQL or Oracle, conforming to at least 170 out of 179 core requirements. It prioritizes strict standards adherence, data integrity, and ACID compliance.

Boringly reliable. Wikipedia, Reddit, Instagram, and even the U.S. Federal Aviation Administration run on Postgres. It is the workhorse of modern data infrastructure.

Extra features. JSONB, full-text search, PostGIS for spatial data, time-series extensions. We'll meet a few of these in Week 12.

It's also what I use at home for my own projects.

Postgres in the Wild

A small slice of who runs production workloads on PostgreSQL:

Wikipedia	Reddit	Instagram	Spotify
Apple	Stripe	GitLab	Robinhood
FAA	IRS	TripAdvisor	Twitch

Plus most of the modern data startups and a meaningful chunk of the U.S. federal government. No one will regret knowing Postgres.

SQL is a transferable skill

What you learn in this class transfers directly to MySQL, Ms SQL Server, Oracle, MariaDB, Snowflake, and any other relational database.

The dialect differs in small details, but the overall model is the same.

SQL is SQL

First contact with Postgres

Pagila - Our In-Class Dataset

Pagila is a sample database

It models a fictional movie rental store.

it's pre-built, has a clean design, and well-populated with data.

You can practice querying without also having to debugging your own schema.

Note that Pagila is the in-class dataset. The homework dataset - your soccer league - you'll design yourself starting next week.

Pagila's Tables

Don't memorize. You'll learn them by writing queries against them.

Films & Content

film
actor
category
language
film_actor
film_category

Inventory & Rentals

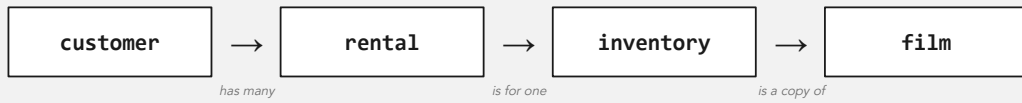
inventory
store
rental
payment

Customers & Staff

customer
staff
address
city
country

22 tables in total once you count the small junction tables. We'll meet most of them in the lab.

How They Connect



And on the side: a film has many actors. A film has one or more categories. A customer lives at an address in a city in a country. A store has a manager (staff).

When you write a query, you're tracing a path through this graph. "What did customer X spend last month" is a walk from customer → payment. "Films featuring actor Y" is film_actor → film.

By Week 5 this graph will be like second nature..

Two Ways to Talk to Postgres

psql

the command-line client

Fast. Scriptable.

Standard on every Postgres server.

Best for: live demos, scripts, automation.

DBeaver

the graphical client

Visual schema browser, click-to-explore.

Result tables you can sort and filter.

Best for: exploring, working through homework.

What I expect you to use most.

Be comfortable with both. They talk to the same Postgres - they're just different windows onto it.

Connecting to Postgres

This string shows up everywhere - DBeaver, psql, Python, env files. Worth knowing the parts.

```
postgresql://dan:secret@localhost:5432/pagila
```

postgresql://	the protocol - always this
dan:secret	username and password (password may be empty for local installs)
localhost	the host running Postgres - your laptop tonight, a server later
5432	the port - Postgres default; you almost never change this
pagila	the database name to connect to

Errors You'll See Tonight

Postgres errors are usually informative. Read them - they almost always tell you the problem.

ERROR: relation "fillm" does not exist

Typo in a table name - or you're connected to the wrong database. Check your spelling, then check `\dt` (psql) or DBeaver's schema browser.

ERROR: column "titel" does not exist

Same idea, one level down. Typo in a column name. Postgres is case-sensitive only when you double-quote, but it's strict about spelling.

ERROR: syntax error at or near ";"

Something is missing right before the semicolon - usually a comma between SELECT items, or a closing parenthesis you forgot.

psql: could not connect to server

Postgres isn't running, or it's on a different port. Start the service and try again.

Live Demo

Lab 1: First Contact

Lab 1 - What You'll Do

1. Download and install Postgres. Confirm it's running on your machine, and connect from DBeaver.
2. Load the Pagila schema and data files (instructions on Canvas) into a new database called pagila.
3. List the tables in the database. There should be 15 (or 22 if payment tables aren't shown as one).
4. Work through the worksheet on Canvas - eight short SELECTs against the film and actor tables.
5. Wave me over if you get stuck.

If install isn't working, sit with someone whose is. Ideally nobody leaves tonight without a working Postgres.

Things That Will Trip You Up

First-time Postgres setup is the hardest part of this course. Most of it never comes back after tonight.

psql: command not found

Postgres is installed but its bin directory isn't on your PATH. Use the full path `/Library/PostgreSQL/18/bin/psql` on Mac, similar on Windows), or fix your PATH per the install guide.

could not connect to server

Postgres isn't actually running. On Mac: open the Postgres app or `brew services start postgresql@18`. On Windows: `services.msc` → PostgreSQL → Start.

DBeaver "hangs" on connect

Almost always the wrong port or wrong host. Should be `localhost` and `5432`. If you used a different port at install time, use that.

pg_restore: error ... database "pagila" does not exist

You have to create the database first: `createdb pagila`. Then restore into it.

Permission denied / role does not exist

Your local Postgres has a default user (often `postgres`, sometimes your own username). The install guide tells you which.

Homework 1

Answer six short answer questions about what we discussed in the lecture.

Write 9 plain SELECT queries against Pagila. Filtering, sorting, limiting only - no joins yet, no aggregation yet. The point is reps and confidence with the basics.

What I want to see: a single .sql file with each query commented ("-- Q1: ..."). Use the template/starter file on Canvas.

Due: start of class next Tuesday, May 19. Submit on Canvas.

Next Week

Week 2 - Entity-Relationship modeling. Before we write more SQL, we'll learn how to look at a domain and turn it into a schema.

We'll meet your homework domain for the rest of the semester: the soccer league we briefly discussed. You will design its schema next week and live with it through Week 11.

Before next class: make sure Postgres and DBeaver are working. Skim the Postgres and Pagila documentation. Submit the lab and get HW1 in.

Thanks for a good first night.

Questions?

Or - see you next Tuesday.